

Объектно-
ориентированное
программирование с
использованием языка C++
ООП

Оглавление

- Глава 1. Введение
- Глава 2. Структуры
- Глава 3. Классы и концепция ООП
- Глава 4. Структура многофайлового проекта
- Глава 5. Использование : в конструкторе
- Глава 6. Инкапсуляция
- Глава 7. Конструктор копирования
- Глава 8. Static члены и методы класса
- Глава 9. Полиморфизм
- Глава 10. Наследование
 - Глава 11. Умные указатели
- Глава 12. Агрегация и композиция
- Глава 13. Rvalue, lvalue, gvalue, pvalue
- Глава 14. Перенос, std::move
- Глава 15. Динамические структуры данных

Глава 1. Введение

- Принцип изучения дисциплины
- Правила написания кода

Принцип изучения дисциплины

- Объектно – ориентированное программирование – парадигма, своеобразный набор «правил», а не список технических особенностей
- Во время обучения будет следующий подход:
 - Сначала теоретическое (философское) объяснение какого-либо подхода или явления
 - Потом, способы реализации данного подхода или явления на языке C++
- Важно! ООП не равно C++ – знания, полученные с теоретической точки зрения будут актуальны на любом другом языке
- Однако, технические особенности будут рассмотрены именно на языке программирования C++ и могут различаться с особенностями других языков

Принцип изучения дисциплины

- Основные правила:
 - Никаких долгов к дате сдачи экзамена
 - Программы надо защищать лично
 - Пропуск занятия не освобождает от выполнения задания
- В данной дисциплине программы могут быть довольно объёмные, так что общее количество их будет меньше, чем в предыдущей дисциплине (Основы программирования на языке C++), но времени на решение будет уходить больше

Правила написания кода

```
1  #include <iostream> //Указывайте ВСЕ библиотеки, необходимые для работы этого файла
2  class MyClass {
3      int number_; //Поля (переменные-члены класса) с _ в конце
4      char character_;
5      std::string string_with_special_name_; //snake_case всё ещё актуален (но с _ в конце)
6      MyClass() {} //Конструктор – специальный метод, по другому его не назвать (CamelCase)
7      void myMethod() {} //Собственный метод (первое слово с маленькой, последующие с большой)
8  };
9  void anotherFunction(int param1, int param2) { //Функции пишутся "первое с маленькой"
10
11  }
12  int main() {
13      int main_num; //У обычных переменных (не полей) _ в конце не ставится
14  }
```

Глава 2. Структуры

- Что такое структуры и откуда они
- Как создать структуру и её экземпляр
- Как работать со структурами
- **Задание 2.1**

Что такое структуры и откуда они

Структуры – совокупность переменных, с собственным именем

Исторически язык программирования С не имел возможности работы с классами и объектами, но в нём была другая возможность – структуры и экземпляры структур

```
1      #include <iostream>
2      struct Human//Есть некоторая структура ЧЕЛОВЕК
3      {
4          std::string name;//У человека есть имя
5          int age;//У человека есть возраст
6      };
7      int main() {
8          //Совокупность двух переменных ниже как бы тоже даёт вам сущность ЧЕЛОВЕК
9          std::string name;//У человека есть имя
10         int age;//У человека есть возраст
11
12         name = "John"; age = 18;//Но приходится работать с разными переменными (можно попутать)
13
14         Human human1;//При создании экземпляра переменные упакованы (прямо как в коробку)
15         human1.name = "Elisa"; human1.age = 22;//Переменные удобно объединены под одним именем
16     }
```


Как создать структуру и её экземпляр

```
1  #include <iostream>
2  //Для создания используется ключевое слово struct
3  struct Car //После struct указывается имя структуры
4  //Содержимое структуры находится внутри фигурных скобок (не забываем про область видимости)
5  {
6      std::string model;
7      int horsepower;
8  }; //В конце структура заканчивается ;
9
10 int main() {
11     int num1; //Создание экземпляра целого числа
12     Car car1; //Создание экземпляра машины
13     std::cin >> num1; //Число можно ввести с клавиатуры
14     std::cin >> car1; //Машину ввести с клавиатуры нельзя
15     std::cin >> car1.model >> car1.horsepower; //Так как машина – составной объект из 2 переменных
16 }
```

Как работать со структурами

Для работы с экземплярами структур используют функции

```
1  #include <iostream>
2
3  struct Cat { //Допустим есть структура, описывающая параметры кошки (кличка, цвет, возраст)
4      std::string name, color;
5      int age;
6  };
7  void editCat(Cat& cat) { //Обратите внимание – экземпляр получаем по ссылке
8      std::cin >> cat.name >> cat.color >> cat.age; //В данной функции почти не получится ошибиться
9  }
10 int main() {
11     Cat cat1, cat2;
12     //Для обращения к переменным у конкретного экземпляра используется точка
13     std::cin >> cat1.name >> cat2.color >> cat1.name; //И так всё ещё можно напутать (cat2 вместо cat1)
14     //Поэтому используют отдельные функции (меньше написанного кода – меньше мест для ошибок)
15     editCat(cat1); //Тут заполнится кошка1
16     editCat(cat2); //Тут заполнится кошка2
17     //Если вдруг кошки неправильно заполнятся – то ошибка в функции (надо проверить одно место)
18 }
```

Задание 2.1

1. Программа запрашивает данные пользователя: имя, фамилию, дату рождения (день, месяц, год)

Вычислить:

- Текущий возраст пользователя (примерно)
- Знак зодиака

Вывести всю полученную и вычисленную информацию на экран

«Человека» реализовать через структуру

```
Текущая кодовая страница: 1251
```

```
Введите своё имя >_:Пётр
```

```
Введите свою фамилию >_:Шоколадка
```

```
Введите день рождения (одно число) >_:19
```

```
Введите месяц рождения (одно число) >_:8
```

```
Введите год рождения (одно число) >_:1985
```

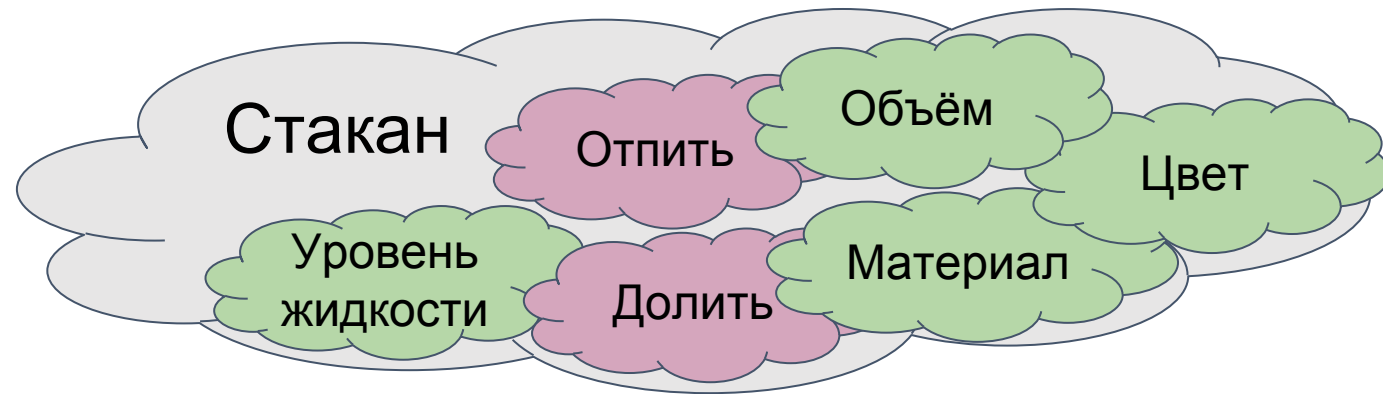
```
Приветствуем вас, Шоколадка Пётр, 19.8.1985 г.р., вам сейчас примерно 41 лет, ваш знак зодиака: Лев
```

Глава 3. Классы и концепция ООП

- Объектно-Ориентированное Программирование
- Классы и объекты
- Было/стало
- Для чего?
- Из чего состоит класс
- Поле, Метод
- Конструктор
- Деструктор
- Задание 3.1
- Задание 3.2
- this
- Задание 3.3

Объектно-Ориентированное Программирование

- Парадигма ООП направлена на упрощение архитектурной разработки и проектирования. Также позволяет облегчить дальнейшую поддержку проекта
- Наглядный пример ООП - реальный мир и человеческое восприятие этого мира
- **ООП - парадигма в программировании, основанная на классах, объектах и их взаимодействии**



Классы и объекты

Класс - совокупность переменных, которые могут описывать объект, и функций, которые позволяют взаимодействовать с этим объектом или этому объекту с другими элементами структуры программы.

Объект - единичный уникальный представитель класса со своим уникальным состоянием (набором значений параметров)



Было/стало

```
1  #include <iostream>
2  struct Apple//Структура Яблоко
3  {
4      std::string color;//Цвет яблока
5      bool is_sour;//Кислое ли яблоко
6  };
7  void editApple(Apple& apple) {//Отдельная функция для заполнения информации о яблоке
8      std::cin >> apple.color >> apple.is_sour;
9  }
10 void printApple(const Apple& apple) {//Отдельная функция для вывода информации о яблоке
11     std::cout << "У яблока цвет " << apple.color << " и оно " << (apple.is_sour ? "кислое" : "не кислое") << std::endl;
12 }
13 int main() {
14     Apple apple;//Создаём яблоко
15     editApple(apple);//вызываем глобальную функцию (доступна всем), куда отправляем нужное яблоко
16     printApple(apple);//вызываем глобальную функцию (доступна всем), куда отправляем нужное яблоко
17 }
```

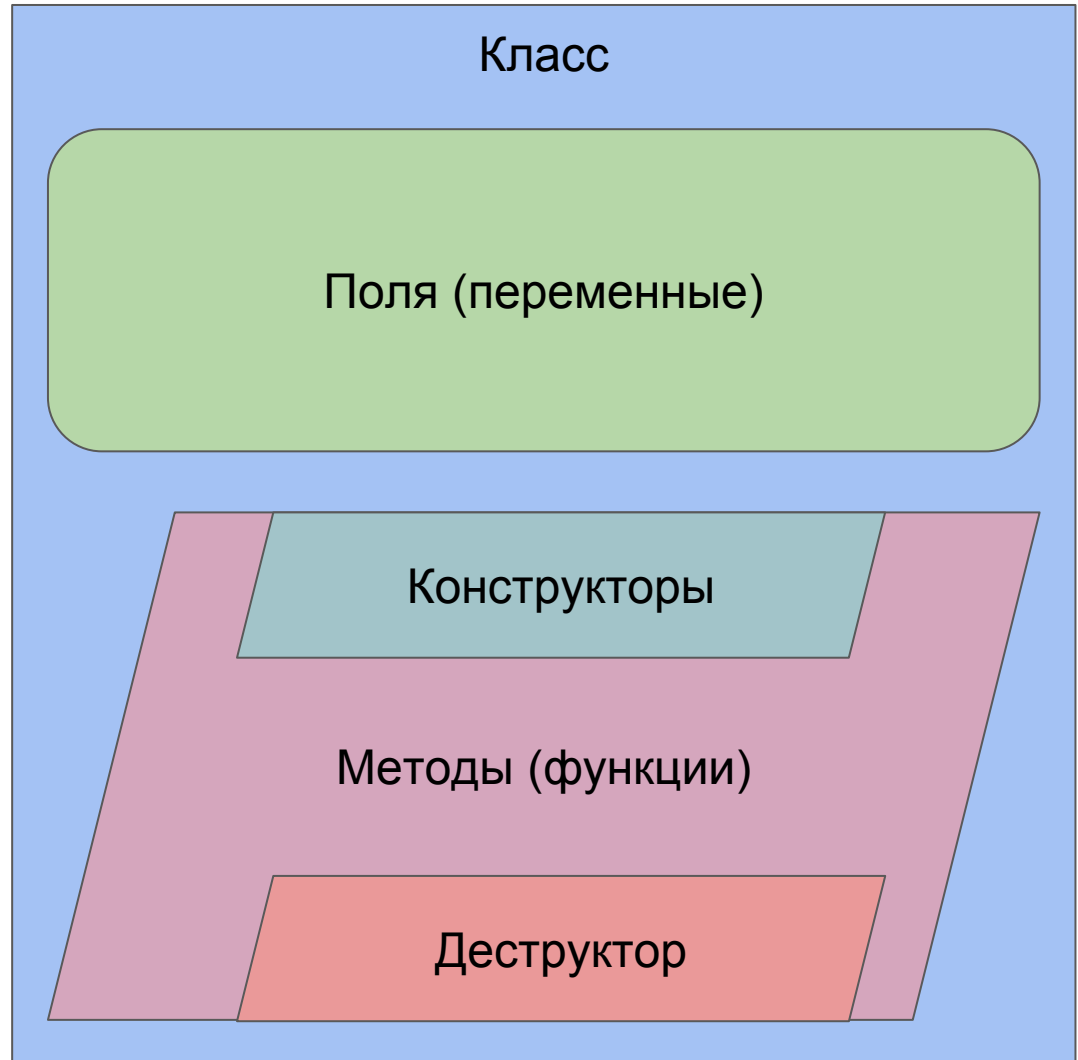
```
1  #include <iostream>
2  class Apple//класс Яблоко
3  {
4      //Поля теперь просто так "снаружи" класса недоступны
5      std::string color;//Цвет яблока
6      bool is_sour;//Кислое ли яблоко
7  public://А вот то, что ниже – доступно (в нашем случае действия над яблоком)
8      void editApple() {//Эта функция изменяет то яблоко, у которого она будет вызвана
9          std::cin >> color >> is_sour;//Так как функции находятся внутри класса – им не надо уточнять по типу apple.
10     }
11     void printApple() {//Эта функция выведет информацию о том яблоке, у которого она будет вызвана
12         std::cout << "У яблока цвет " << color << " и оно " << (is_sour ? "кислое" : "не кислое") << std::endl;
13     }
14 };
15 int main() {
16     Apple apple, apple1;//Создаём несколько яблок
17     apple.editApple();//вызываем действие у конкретного яблока (изменим конкретно apple)
18     apple1.editApple();//вызываем действие у другого яблока (изменим конкретно apple1)
19     apple.printApple();//функция выведет информацию о яблоке apple
20 }
```

Для чего?

- Язык C++ разрабатывался с учётом обратной совместимости с языком C (код на C должен был запускаться на C++) – так происходит и по сей день
- На C было написано очень много полезных и эффективных библиотек
- Это позволяет C++ при желании опуститься до языка C и начать работать на ещё более низком уровне абстракции (более эффективно по отношению к машине)

Из чего состоит класс

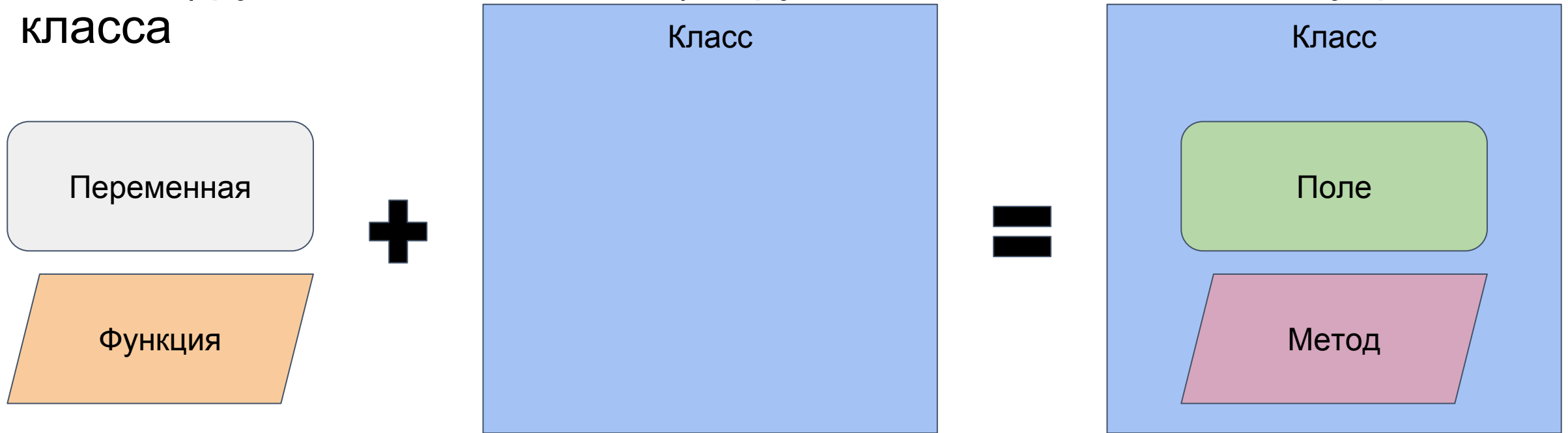
- Поля
- Методы
- Конструкторы
- Деструктор



Поле, Метод

Поле (переменная-член класса) - переменная, находящаяся внутри класса

Метод (функция-член класса) - функция, находящаяся внутри класса

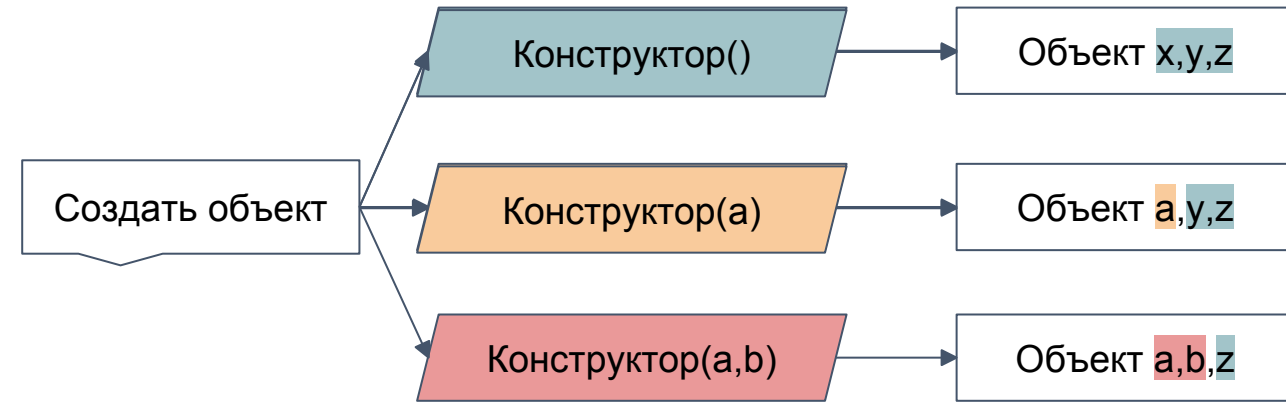


Конструктор

Конструктор - метод, который выполняется при создании объекта класса

Конструктор может принимать значения в качестве списка параметров, а также может быть перегружен

Конструктор используется, например, для задания начальных значений переменным



```
1  #include <iostream>
2  class MyClass {
3      int num_; // Допустим в классе есть поле
4  public: // Конструкторы должны быть публичными (пока что всегда)
5      MyClass() { // Это конструктор по умолчанию (пустой список перед. парам.)
6          num_ = 0; // Просто обнуляем переменную
7      }
8      MyClass(int num) { // Пример перегрузки конструктора - этот уже принимает значение
9          num_ = num; // Которое и запишем в наше поле
10     }
11     void printNum() { // У простой функции, в отличие от конструктора есть тип возвр. знач.
12         std::cout << num_ << std::endl;
13     }
14 };
15 int main() {
16     MyClass obj1; // Тут сработает конструктор по умолчанию
17     MyClass obj2(77); // Тут сработает конструктор от целого значения
18     MyClass obj3('a'); // Тут кстати тоже :)
19     obj1.printNum();
20     obj2.printNum();
21 }
```

Деструктор

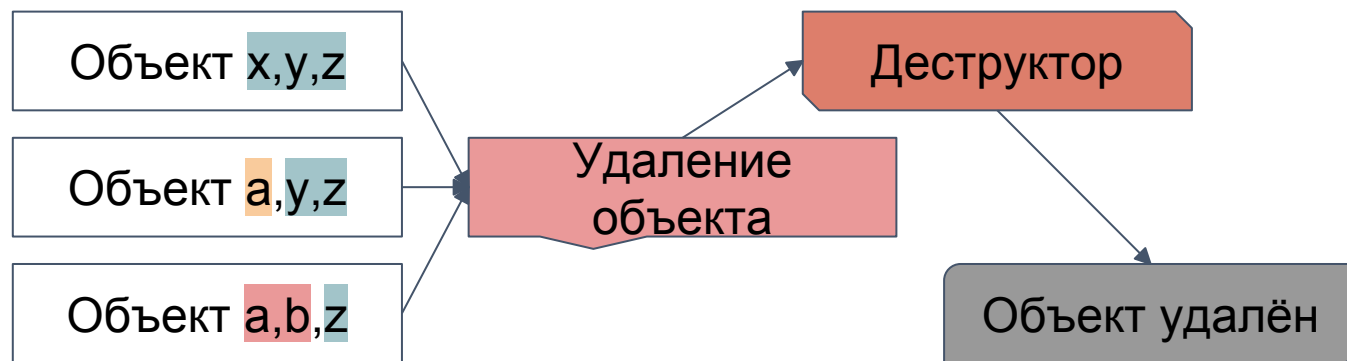
При выделении динамической памяти, или при работе со сторонними ресурсами (например, файловой системой), необходимо эту самую память освободить или “отвязывать” ресурсы.

Деструктор - специальный метод, который вызывается при удалении объекта класса и действует только на этот объект.

Деструктор не принимает параметров, не возвращает значений и не может быть перегружен

Деструктор может быть только один!

Не важно, как объект был создан – удалён он будет единственным способом



```
1  #include <iostream>
2  class MyClass {
3      int* ptr;
4  public: //Деструкторы тоже должны быть доступны
5      MyClass() {
6          ptr = new int(0);
7          std::cout << "Объект был создан\n";
8      }
9      ~MyClass() { //Деструктор отличается от конструктора ~ в начале
10         delete ptr;
11         ptr = nullptr;
12         std::cout << "Объект был удалён\n";
13     }
14 };
15 int main() {
16     {
17         MyClass obj1; //Тут объект начнёт существовать
18     } //В конце области видимости сработает деструктор и освободит память
19 }
```

Задание 3.1

Создать класс, описывающий автомобиль Car:

- **Поля:** марка, пробег, год выпуска, страна - производитель, мощность

Программа должна запросить у пользователя количество вводимых автомобилей

После чего пользователь вводит данные о них

По окончании заполнения вывести данные о всех автомобилях на экран, отсортировав их по году выпуска

```
Текущая кодовая страница: 1251
Введите количество автомобилей: 3
Введите год >_:2006
Введите марку >_:Mazda RX-8
Введите пробег >_:500000
Введите страну-производитель >_:Япония
Введите мощность (в лошадиных силах) >_:140
```

```
Введите год >_:1967
Введите марку >_:Ford Mustang
Введите пробег >_:400000
Введите страну-производитель >_:США
Введите мощность (в лошадиных силах) >_:122
```

```
Введите год >_:2022
Введите марку >_:УАЗ БУХАНКА
Введите пробег >_:1000000
Введите страну-производитель >_:Россия
Введите мощность (в лошадиных силах) >_:1
```

Машина:

```
Ford Mustang
Производитель: США
Мощность: 122 л.с.
1967 года выпуска
пробег: 400000км.
```

Машина:

```
Mazda RX-8
Производитель: Япония
Мощность: 140 л.с.
2006 года выпуска
пробег: 500000км.
```

Машина:

```
УАЗ БУХАНКА
Производитель: Россия
Мощность: 1 л.с.
2022 года выпуска
пробег: 1000000км.
```


Задание 3.2

Создать класс **Student**, описывающий студента:

- **Поля:** имя, возраст, идентификатор, класс обучения
- **Методы:** редактирование студента, вывод информации, конструктор(ы), деструктор

main():

- ❖ Запросить у пользователя кол-во студентов
- ❖ Создать массив студентов необходимого размера
- ❖ При создании объекта класса Student выводить сообщение о его создании (объекта)
- ❖ При удалении объекта класса Student выводить сообщение о его удалении, при этом указывая имя удаляемого студента
- ❖ Организовать простое меню для пользователя:
 1. Отредактировать студента
 2. Выйти из программы
- ❖ При работе программы список студентов должен постоянно обновляться автоматически и выводиться сверху (над меню)

Текущая кодовая страница: 1251
1 Введите количество студентов: 3
Студент был создан!
Студент был создан!
Студент был создан!

2
0.Студент #-1
unknown
0 лет
null
1.Студент #-1
unknown
0 лет
null
2.Студент #-1
unknown
0 лет
null
1. Отредактировать студента
2. Выход
>_:1

4
0.Студент #123
Николай
18 лет
11 Ж
1.Студент #-1
unknown
0 лет
null
2.Студент #-1
unknown
0 лет
null
1. Отредактировать студента
2. Выход
>_:

3
0.Студент #-1
unknown
0 лет
null
1.Студент #-1
unknown
0 лет
null
2.Студент #-1
unknown
0 лет
null
1. Отредактировать студента
2. Выход
>_:1
Введите порядковый номер студента из списка выше (не id) >_:0

Введите уникальный номер студента: 123
Введите имя студента >_:Николай
Введите возраст студента >_:18
Введите класс обучения >_:11 Ж

5
Всего доброго!
Студент Николай(#123) был удалён!
Студент Жанна(#321) был удалён!
Студент Лёха(#303) был удалён!

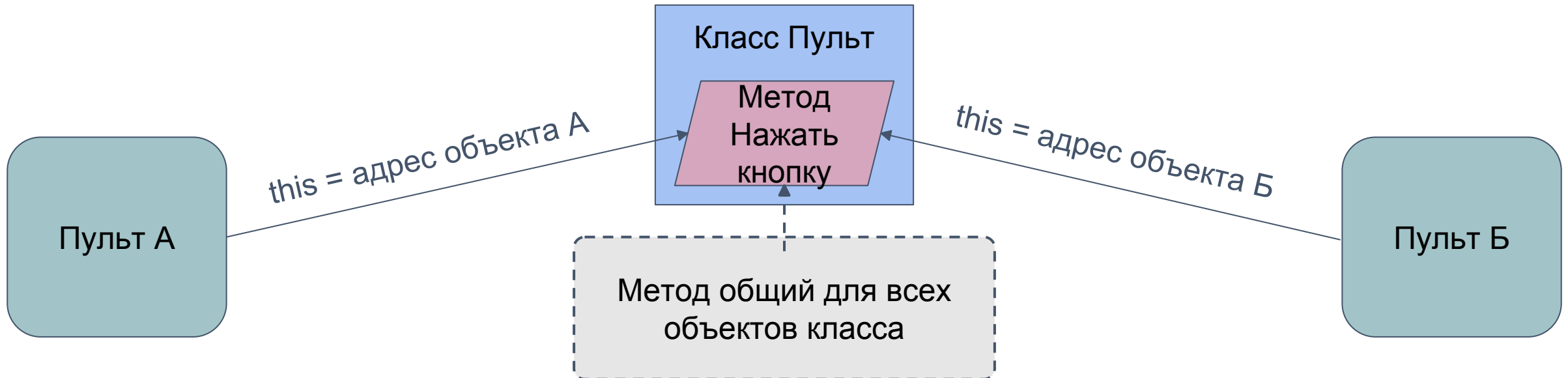
this

this (англ. этот) - указатель на текущий экземпляр класса.

this можно использовать для ликвидации проблемы совпадающих имён (в классе есть x, передаём в метод тоже x)

также this можно использовать для идентификации объекта (по адресу)

```
1  #include <iostream>
2  class MyClass {
3      int num; //Если у нас есть num
4  public:
5      MyClass(int num) { //И мы передаём num (переданный num)
6          num = num; //То тут мы в переданный num запишем его же значение
7          this->num = num; //А вот тут мы уточнили и в num из 3 строки запишем переданный
8      }
9      void showAddress() { //Есть ещё одно применение ключевого слова this:
10         std::cout << "У этого объекта адрес " << this << std::endl; //Можно вывести адрес
11     }
12
13 };
14 int main() {
15
16 }
```



Задание 3.3

Создать класс, описывающий планету, **Planet**:

- Если возникают проблемы с составлением параметров, то можно подсмотреть на сайте: [ссылка](#)
- Копировать всё оттуда не обязательно!

main():

- ★ Создать массив планет, организовав тем самым систему планет (например, Солнечную)
- ★ Информацию о системе надо вывести на экран
- ★ При создании массива, его элементы можно проинициализировать через конструктор с параметрами, который вы можете написать в классе

Меркурий:

Масса 0.055 масс Земли
Состав атмосферы: -
Давление у поверхности: 0
Количество спутников: 0

Венера:

Масса 0.815 масс Земли
Состав атмосферы: углекислый газ, азот, вода
Давление у поверхности: 90
Количество спутников: 0

Земля:

Масса 1 масс Земли
Состав атмосферы: азот, кислород, аргон
Давление у поверхности: 1
Количество спутников: 1

Марс:

Масса 0.38 масс Земли
Состав атмосферы: углекислый газ, азот, аргон
Давление у поверхности: 0.006
Количество спутников: 2

Юпитер:

Масса 317.8 масс Земли
Состав атмосферы: водород, гелий
Давление у поверхности: -1
Количество спутников: 62

Сатурн:

Масса 95.5 масс Земли
Состав атмосферы: водород, гелий
Давление у поверхности: -1
Количество спутников: 34

Уран:

Масса 14.37 масс Земли
Состав атмосферы: водород, гелий, метан
Давление у поверхности: -1
Количество спутников: 27

Нептун:

Масса 17.15 масс Земли
Состав атмосферы: водород, гелий, метан
Давление у поверхности: -1
Количество спутников: 13

Жока:

Масса 0.5 масс Земли
Состав атмосферы: спирт
Давление у поверхности: 0
Количество спутников: 2

Тракторист:

Масса 300 масс Земли
Состав атмосферы: белое машинное масло
Давление у поверхности: 300
Количество спутников: 0

Шутки про твою маму:

Масса 1e+08 масс Земли
Состав атмосферы: белое машинное масло
Давление у поверхности: 10000
Количество спутников: 999

Глава 4. Структура многофайлового проекта

- Механика «выноса» методов за класс
- Пример отдельной инициализации
- **Задание 4.1**
- Типы файлов
- Что в каких файлах писать
- Пример «архитектуры»
- Как правильно/неправильно делать
- **Задание 4.2**
- Команды препроцессора
- Некоторые команды препроцессора
- Для чего используются команды препроцессора
- **Задание 4.3**

Механика «выноса» методов за класс

Принцип почти такой же, как и при раздельном объявлении и реализации функций:

- Объявление ограничить сигнатурой метода
- Реализацию написать отдельно от класса (за его пределами)
- Есть нюанс: полное имя метода идёт с указанием класса с использованием двойного двоеточия (как с `std::`)
- Содержимое всё так же пишется внутри фигурных скобок

Пример отдельной инициализации

```
1  #include <iostream>
2  class MyClass {
3  |   char character_; //поле для "полезной нагрузки" класса
4  | public:
5  |   MyClass() { //Конструктор, объявленный и реализованный в классе
6  |       character_ = 'Ъ';
7  |   }
8  |   ~MyClass(); //Деструктор, ТОЛЬКО ОБЪЯВЛЕННЫЙ в класса
9  |   void input() { //Метод, объявленный и реализованный в классе
10 |       std::cin >> character_;
11 |   }
12 |   void output(); //Метод, ТОЛЬКО ОБЪЯВЛЕННЫЙ в классе
13 | };
14 | void MyClass::output() { //Реализация метода за пределами класса
15 |     std::cout << character_ << '\n';
16 | }
17 | MyClass::~MyClass() { //Реализация конструктора за пределами класса
18 |     character_ = ' ';
19 | }
20 | int main() {
21 |     system("chcp 1251");
22 |     MyClass obj; //Конструктор работает
23 |     obj.output(); //Вывод работает
24 |     obj.input(); //Ввод работает
25 |     obj.output(); //Вывод работает
26 |     obj::~MyClass(); //Смысла не имеет, но сам факт запуска зафиксирован
27 | }
```

Консоль отладки Microsoft Visual Studio

Текущая кодовая страница: 1251

Ъ
]
]
]

Задание 4.1

Написать класс, описывающий кровать **Bed**:

- Поля: двуярусная (да/нет), количество мест (односпальная, двухспальная), материал (метал, пластик, дерево и т.п.), стоимость
- Методы: конструктор(ы), редактирование, вывод информации

main():

- Запросить у пользователя количество кроватей
- Спросить у пользователя: будет ли он заполнять информацию о каждой кровати сам или будут кровати «по умолчанию»
- Создать необходимое количество кроватей с выбранным способом создания и вывести информацию о них на экран

❖ **РЕАЛИЗАЦИЮ МЕТОДОВ И КОНСТРУКТОРОВ КЛАССА ВЫНЕСТИ ЗА ПРЕДЕЛЫ КЛАССА**

❖ **ПОЛЯ ИНИЦИАЛИЗИРОВАТЬ ТОЛЬКО В КОНСТРУКТОРАХ**

Текущая кодовая страница: 1251

Введите количество кроватей >_:3

Заполните сами? y/n >_:n

однярусная кровать из Метал, рассчитана на 1 человек, стоимость: 15001

однярусная кровать из Метал, рассчитана на 1 человек, стоимость: 15001

однярусная кровать из Метал, рассчитана на 1 человек, стоимость: 15001

Текущая кодовая страница: 1251

Введите количество кроватей >_:2

Заполните сами? y/n >_:y

Двуярусная кровать? y/n >_:y

Из какого материала кровать? >_:дерево

На сколько человек кровать?(целое значение) >_:2

Сколько стоит данная кровать?(можно дробное) >_:40500.99

Двуярусная кровать? y/n >_:n

Из какого материала кровать? >_:хлебный мякиш

На сколько человек кровать?(целое значение) >_:4

Сколько стоит данная кровать?(можно дробное) >_:123312.01

двуярусная кровать из дерево, рассчитана на 2 человек, стоимость: 40501

однярусная кровать из хлебный мякиш, рассчитана на 4 человек, стоимость: 123312

Типы файлов

При разработке программ на языке программирования C++ используются 2 типа файлов:

- Заголовочные (.h или .hpp) – это исходный код библиотек
- Исполняемые (.cpp) – это исходный код программ

Но если рассматривать распределение объёма написанного кода на различные (выше указанные) файлы, то получается следующая логика:

- В заголовочных указывается скелет класса
- В исполняемых указанные части из заголовочного получают своё наполнение

Что в каких файлах писать

Исполняемые файлы (.cpp)

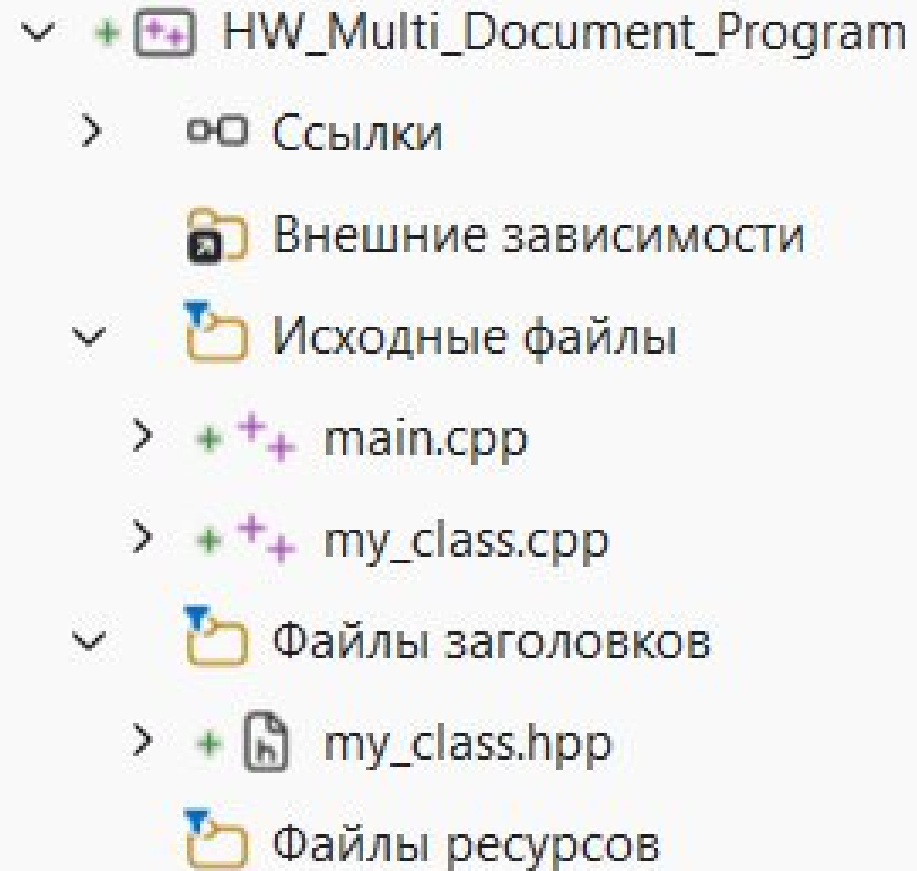
- Весь код, в котором происходят вычисления
- Также можно подключать заголовочные файлы и библиотеки

Заголовочные файлы (.h)

- Объявления классов и функций
- Подключение библиотек для корректной работы заголовочного файла

Пример «архитектуры»

- `main.cpp` – центральный файл. В нём будет `int main()` и собираются все классы для взаимодействия (не обязательно)
- `my_class.cpp` – исполняемый файл вашего класса. В нём пишутся реализации методов, указанные в заголовочном файле
- `my_class.hpp` (или `.h`) – заголовочный файл. В нём объявляются классы и методы

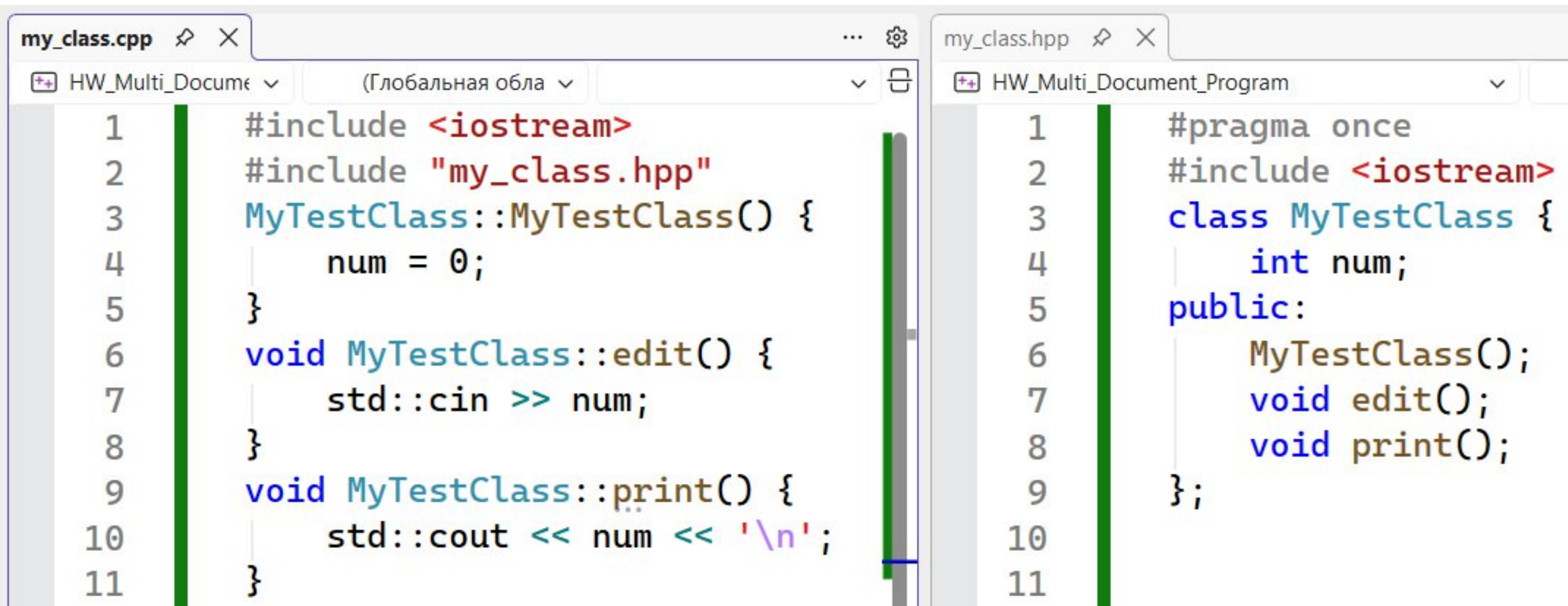


Неправильно

- В заголовочном файле есть исполняемый код
- Методы надо только объявить

```
my_class.hpp  ✎  ✕  
HW_Multi_Document_Program  (Глобальная с  
1  #pragma once  
2  #include <iostream>  
3  class MyTestClass {  
4      int num;  
5  public:  
6      MyTestClass() {  
7          num = 0;  
8      }  
9      void edit() {  
10         std::cin >> num;  
11     }  
12     void print() {  
13         std::cout << num;  
14     }  
15 };
```

Правильно



```
my_class.cpp
1  #include <iostream>
2  #include "my_class.hpp"
3  MyTestClass::MyTestClass() {
4      num = 0;
5  }
6  void MyTestClass::edit() {
7      std::cin >> num;
8  }
9  void MyTestClass::print() {
10     std::cout << num << '\n';
11 }

my_class.hpp
1  #pragma once
2  #include <iostream>
3  class MyTestClass {
4      int num;
5  public:
6      MyTestClass();
7      void edit();
8      void print();
9  };
10
11
```

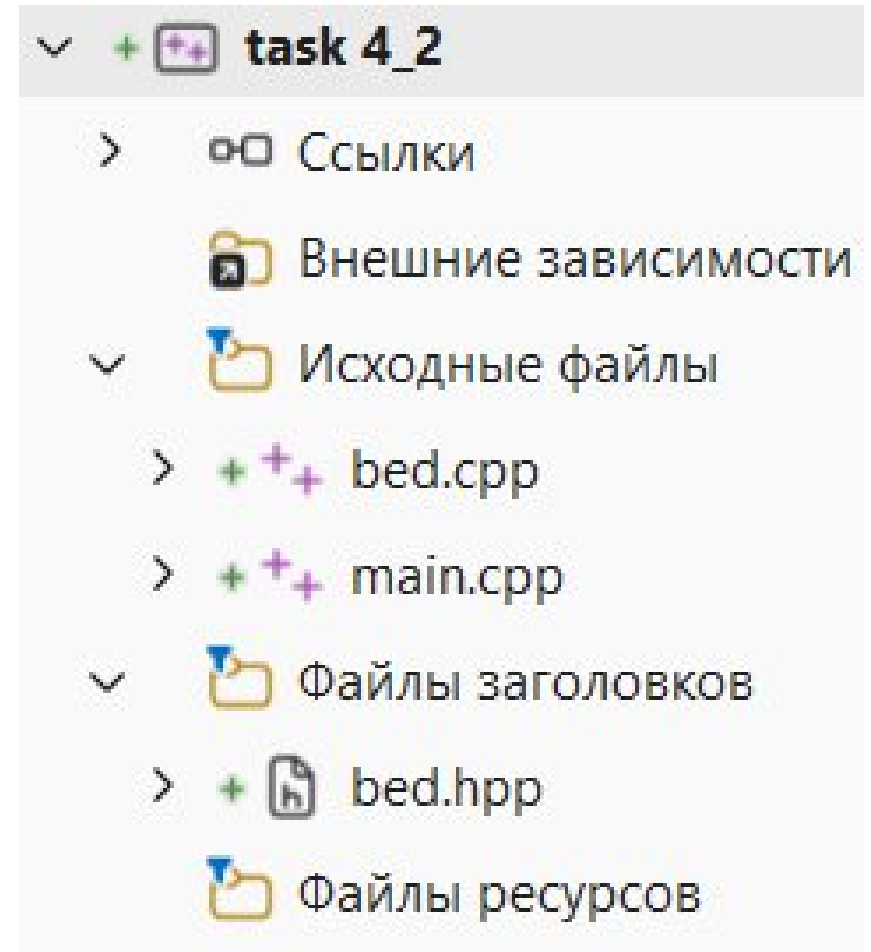
Задание 4.2

Возьмите код Задания 4.1 и распределите его по разным файлам

По итогу должны получиться файлы:

- main.cpp (запрос количества кроватей и работа с множеством кроватей)
- Bed.hpp (описание кровати)
- Bed.cpp (реализация кровати)

Итоговая работа программы не должна измениться (работает точно так же)



Команды препроцессора

- Команды препроцессора (они же макросы) – команды, начинающиеся с **#** в начале документа (`#include` относится к ним), которые срабатывают на самом раннем этапе обработки программного кода компиляторами и линковщиками
- Необходимы команды препроцессора для специфичной работы, выходящей за рамки простого написания кода.

Некоторые команды препроцессора

```
1  #pragma once//данная команда обеспечит одинарное подключение документа во всём проекте (нет дублирования)
2
3  #ifndef PREPROCESSOR//Если во всех документах нет переменной PREPROCESSOR
4  #define PREPROCESSOR//То мы объявляем переменную PREPROCESSOR (теперь она будет)
5  //Код ниже будет добавлен в место вызова (если текущий документ будет где-то подлючен через #include)
6
7  #include <iostream>//Вставит вместо этого вызова 75 строк кода из библиотеки iostream
8  #define SOMETHING1//Создание переменной для ТЕКСТОВОГО ДОКУМЕНТА (не для кода и она не имеет типа)
9  #define OKAK std::cout<<"okak";//При этом в такой переменной может храниться целый код
10
11  int main() {
12      OKAK//При вызове этого макроса подставится его содержимое
13  }
14
15  #endif//Тут завершается работа #ifndef
```

Для чего используются команды препроцессора

- Для контроля областей документа (`#ifndef`)
- Для возможности подмены ключевых слов (`#define`)
- Для подключения других документов (`#include`)
- Для защиты от многократного подключения документа (`#pragma once` или связка `#ifndef` и `#define`)

Задание 4.3

Для выполнения данного задания необходимо выполнения задания 4.2

Защитите свой документ `bed.hpp` от многократного подключения, используя метод `#ifndef` и `#define`

Обратите внимание, что скорее всего у вас уже автоматически используется `#pragma once` – её удалять не надо

Глава 5. Использование : в конструкторах

- Двоеточие для списка инициализации конструктора
- Пример необходимости использования списка инициализации
- Когда использовать список инициализации?
- Как было раньше/как надо сейчас
- **Задание 5.1**
- Двоеточие для делегирования конструкторов
- Пример делегирования
- **Задание 5.2**

Двоеточие для списка инициализации конструктора

Прямая инициализация – вызов конструктора при создании объекта.

Список инициализации конструктора – действия по определению состояния объектов до выполнения тела конструктора.

Список инициализации позволяет дать начальные значения до начала тела конструктора.

Пример необходимости использования списка инициализации

```
1  #include <iostream>
2  class TestClass {
3      int test_num_; //Есть целочисленная переменная
4      int& test_link_; //Есть ссылка на целочисленную переменную (удивительно – она может быть пустой)
5  public:
6      TestClass() { //Но на этапе написания конструктора будет ошибка, которая даже не позволит начать его тело
7          test_link_ = test_num_; //проблема в том, что ссылка не может быть пустой при объявлении
8          test_num_ = 0; //Тело конструктора начнёт выдавать нетипичные ошибки (нам не дадут доступ к int)
9      }
10     //А вот в конструкторе ниже происходит прямая инициализация, которая позволяет "включиться" перед телом
11     TestClass() :test_num_(0), test_link_(test_num_){ //Ссылка получит свой объект до начала тела конструктора
12         test_num_ = 11; //Даже int опять заработал!
13     }
14 };
15 int main() {
16     int& main_test_link; //Обратите внимание, что мы не можем (и раньше не могли) создавать "пустых" ссылок
17 }
```

Когда использовать список инициализации?

- По сути список инициализации желательно использовать всегда (помимо случая, который будет описан в этой же главе)
- Он позволяет дать начальные значения там, где не может с этим справиться тело конструктора
- Есть мнение, что список инициализации работает быстрее (лично не проверено)

Как было раньше/как надо сейчас

```
1 #include <iostream>
2 //Так делать можно, но лучше от этого отходить (в этом примере ошибок не будет)
3 class MyClass {
4     int num_;
5     std::string str_;
6 public:
7     MyClass() {
8         num_ = 0;
9         str_ = "amogus";
10    }
11    MyClass(int num, std::string str) {
12        num_ = num;
13        str_ = str;
14    }
15 };
16 //Лучше делать так, даже если "вылезут" нестабильные или сложные объекты (по типу ссылок) – будет работать
17 class MyClass1 {
18     int num_;
19     std::string str_;
20 public:
21     MyClass1() :num_{ 0 }, str_("sus") {}//Разницы между {} и () нет. Тело конструктора (даже пустое) после списка инициализации должно быть всегда!
22     MyClass1(int num_, std::string str) :num_{ num_ }, str_{ str } {}//Специально допущена "ошибка" с num_: список сам разберётся при совпадении имён
23 };
```

Задание 5.1

Написать класс, описывающий плитку, **Plate**:

- Поля: высота, ширина, толщина, вес, материал, производитель, стоимость
- Методы: конструктор по умолчанию, конструктор с полными параметрами, вывод информации на экран
- В конструкторах обязательно использовать список инициализации

В **main()** реализуется список доступных плиток в магазине:

- ❖ Создать список из 3-5 плиток заранее (в коде, не руками)
- ❖ Организовать меню для работы со списком
 - Добавить плитку (пользователь вводит данные)
 - Убрать плитку (по индексу из списка)
- ❖ Текущее состояние списка выводить над меню (обновлять при изменении)

```
0.Плитка из керамика; размеры (в ш г) в мм.: 220, 150, 10; вес: 500 грамм, стоимость: 89 рублей, производитель: Моя плитка
1.Плитка из керамика, стекло; размеры (в ш г) в мм.: 220, 150, 10; вес: 450 грамм, стоимость: 150 рублей, производитель: Моя плитка
2.Плитка из бетон; размеры (в ш г) в мм.: 400, 200, 50; вес: 1500 грамм, стоимость: 550 рублей, производитель: STAFK
3.Плитка из стекло; размеры (в ш г) в мм.: 50, 50, 5; вес: 300 грамм, стоимость: 89 рублей, производитель: Элита
1. Добавить плитку
2. Удалить плитку
>_:
```

Двоеточие для делегирования конструкторов

Ещё одна возможность использования двоеточия – делегирование конструкторов

Делегирование конструкторов – процесс вызова одним конструктором другого

Для делегирования используется то же пространство в коде, что и для списка инициализации – следовательно, использовать их одновременно не получится



Пример делегирования

- Делегирование позволяет уменьшить объём кода
- При делегировании именно что вызывается другой конструктор
- Использовать одновременно список инициализации и делегирование нельзя

```
1  #include <iostream>
2  class Test {
3      int num_;
4      char ch_;
5      bool b_;
6  public:
7      Test() :num_{ 0 }, ch_{ '-' }, b_{ false } {}//В конструкторе по умолчанию используется список инициализации
8      Test(int num) :Test() { num_ = num; }//Тут вместо списка инициализации используется делегирование (вызовется Test())
9      Test(int num, char ch) :Test(num) { ch_ = ch; }//Тут мы вызовем предыдущий конструктор (как в эстафете передадим ему параметр num)
10     Test(int num, char ch, bool b) :Test(num, ch) { b_ = b; }//Тут аналогичным образом вызовем предыдущий конструктор (хотя по сути можно любой)
11 };
12 int main() {
13     Test t1,
14         t2(11),
15         t3(-1, 'a'),
16         t4(101, 'b', true);
17 }
```

Задание 5.2

Создать класс (описывающий рабочего) **Worker**:

- **Поля:** id (не редактируемый, может быть задан только при создании), name (фамилия, имя, отчество), пол, возраст, должность, отдел
- **Методы:** Вывод на экран, создать конструкторы для каждого из возможных вариантов инициализации (ФИО; ФИО+пол; ФИО+пол+возраст; и т.д.). **Необходимо** использовать делегирование конструкторов

main():

- ❖ Создать массив (вектор) рабочих
- ❖ Организовать меню для работы с вектором рабочих:
 - Вывести на экран
 - Редактировать рабочего
 - Добавить рабочего
 - Удалить рабочего

Глава 6. Инкапсуляция